DocSort: A Review-Centered Workflow for OCR-Assisted Document Management

Debashish Kanjalkar, Abhay Thombre, Prajwal Joshi, Rahul More
Institute of Information and Communication Technology, MGM University
Chhatrapati Sambhajinagar, Maharashtra, India
Project Mentor: Dr. Jaykumar Shreeram Dhage

***Abstract*—A scanned document may be stored safely and still be difficult to find, especially when its filename reveals little about its contents. Optical character recognition (OCR) makes the text available, but a useful document record also needs a category, structured fields, and a way to correct mistakes. This paper describes DocSort, a working academic prototype that connects these tasks in one application. A Flutter client handles file intake and review, while a FastAPI backend validates PDFs and images, runs Tesseract OCR, and optionally uses a Gemini model to suggest document types and fields. Supabase PostgreSQL stores metadata, and private object storage retains the original files. The key design decision is to keep processing completion separate from user verification: every proposed record requires review before it is marked verified. The team reports functioning upload, review, cabinet, search, and export workflows, alongside regression-test results documented in the repository. Some sample documents needed category or field corrections. Because no labeled evaluation set, independent OCR accuracy test, or controlled timing experiment was completed, the paper does not claim predictive superiority or measured time savings. We describe the implementation, its safeguards and limitations, and an evaluation procedure that would allow those questions to be answered in future work.**

***Index Terms*—document management, optical character recognition, generative AI, human-in-the-loop, document classification, Flutter, FastAPI**

I. Introduction

Digital records often accumulate as scanned images, receipts, invoices, certificates, resumes, and reports. Their filenames may be generated automatically, while the information that makes a record useful—such as its type, issue date, or subject—is hidden inside the content. A conventional directory permits storage, but users still need to inspect and manually categorize files when reliable metadata is absent. Image-only PDFs compound the problem because their text is not directly available to ordinary text search.

Optical character recognition (OCR) recovers text, but recognition alone does not decide what kind of document was scanned or which text spans represent meaningful fields. A downstream interpreter can help, but its predictions may be incorrect, particularly for unfamiliar layouts or low-quality scans. For practical record keeping, an extracted value must be both retrievable and correctable. DocSort addresses this full file-to-record lifecycle rather than treating OCR output as the final product.

The paper has three concrete contributions. First, it documents a modular OCR-plus-optional-AI architecture for PDF/image intake, private original-file retention, metadata persistence, and subsequent retrieval. Second, it describes a human-in-the-loop (HITL) state model that deliberately distinguishes machine proposals from user-confirmed records. Third, it presents implementation-oriented evidence and a transparent validation boundary, together with measurable criteria for future formal evaluation. *The work does not introduce a new OCR model or claim superior predictive accuracy over existing systems.*

Our discussion follows three practical questions: (RQ1) can a single application connect input admission, OCR, metadata review, and retrieval; (RQ2) can the application avoid conflating machine suggestions with reviewed records; and (RQ3) what evidence is needed to progress from a successful demonstration to quantitative reliability claims? RQ1 and RQ2 are examined through implementation and functional evidence; RQ3 is addressed by a prospective evaluation protocol rather than invented measurements.

II. Background and Related Work

Tesseract is a widely used OCR engine; Smith describes its recognition architecture and adaptive classification approach [1]. Yet scanned-form interpretation needs more than characters. The FUNSD benchmark highlights entity labeling and linking challenges in noisy scanned forms [2], while LayoutLM jointly represents textual and layout cues for document understanding [3]. Transformer-based TrOCR explores a learned recognition pipeline [4]; Donut instead investigates end-to-end document understanding without a separate OCR engine [5].

These are meaningful alternatives, not components implemented in DocSort. The present prototype instead uses a modular sequence: Tesseract first produces recognized text and an OCR confidence signal, and an optional generative model proposes document categories and fields. This design facilitates inspection of intermediate output but inherits OCR errors and does not explicitly reproduce the layout modeling of LayoutLM or Donut.

Human oversight is a central design choice rather than an afterthought. Research on machine-assisted technical-document annotation describes how automated suggestions can support, rather than replace, human categorization [6]. For DocSort, the operational implication is simple: machine confidence and human verification must remain distinct.

III. System Requirements and Architecture

A. Requirements and Scope

The core system must (i) authenticate a user; (ii) accept common PDFs and image formats; (iii) validate and track processing; (iv) recognize text and optionally propose type-specific fields; (v) allow corrections; and (vi) organize, search, retrieve, and export authorized records. Source files must remain associated with their structured metadata. User access to records must be constrained by backend authorization instead of client-side navigation alone.

The original application concept included departmental handling. The project report emphasizes personal document management as the most clearly demonstrated use case, while the public repository also contains administrative, departmental, secure-sharing, and experimental relationship-mapping features [11]. These extended features are not treated as fully evaluated enterprise capabilities in this paper.

B. Component Architecture

Fig. 1 shows the trust boundaries. The Flutter client provides document selection, progress, review, cabinet views, and search. FastAPI owns file admission, identity and document authorization, upload orchestration, OCR and AI integration, and persistence. The original file is retained in private Supabase Storage; metadata and extracted fields reside in PostgreSQL. AI credentials and privileged storage access remain on the backend rather than in the client.

C. Data Model

The database separates users, upload jobs, document records, structured fields, and folders. A document references an owner, an original-file storage location, its category and review status, and zero or more extracted fields. Upload jobs track processing separately from the saved record. Secondary entities include controlled share links and activity records. This separation permits updates to interpreted metadata without overwriting the original uploaded bytes. Fig. 2 illustrates the primary entities. The diagram is deliberately selective: access policies, indexes, status constraints, and optional organization-specific tables are not fully represented by these arrows.

A document's lifecycle has two independent dimensions. The upload job describes whether processing finished or failed; the document record describes whether a human has confirmed its interpreted metadata. Combining the two into a single status would incorrectly imply that a finished OCR job produced a trustworthy document. Separating them also makes it possible to re-open unreviewed records in the cabinet without losing source-file provenance.

IV. Processing and Human Review

A. Admission, OCR, and Interpretation

For each input, the API checks the filename, extension, recognizable file signature, readable PDF/image structure, and configured size/page bounds. Accepted uploads receive a job identifier, enabling a client to poll progress without holding the initial HTTP request open. The repository documents queued, processing, completed, and failed paths, with a process-local limit on active processors [11].

TABLE I
Core implementation responsibilities

Module	Responsibility
Flutter client	Intake, session/navigation, progress, correction form, cabinet, search, analytics.
FastAPI	Admission, authentication, job coordination, document permissions, export.
Tesseract + optional Gemini	OCR text; suggested type and structured fields.
Supabase	PostgreSQL metadata and private original-file storage.

For image inputs, Tesseract operates on the supplied image. For PDFs, PyMuPDF renders pages, then OCR is applied to the accepted pages and combined for downstream use. A per-word confidence aggregate describes the OCR engine's estimate of text recognition; it is *not* empirical correctness against a reference transcript. When available, Gemini receives bounded OCR text and a restricted set of page images to propose document categories and typed fields. The service validates structured responses before saving them; when AI is unavailable, OCR and manual review remain possible.

B. Verification State Machine

All newly interpreted records enter *Needs Review*. An authenticated reviewer can inspect and correct the category and field values before explicitly confirming the document as *Verified*. The report does not identify an automatic model-retraining loop from these changes. Fig. 3 distinguishes job completion from document verification.

A useful formal distinction is

$$V(d) = \begin{cases} 1, & \text{after an explicit authorized review action,} \\ 0, & \text{for an unreviewed machine proposal,} \end{cases} \tag{1}$$

where $V(d)$ is a workflow state, not a probability that all fields are correct. OCR confidence C_{OCR} and proposed filing confidence C_{AI} are separately stored signals; neither implies $V(d) = 1$.

V. Implementation and Safeguards

A. Application Modules

The Flutter frontend uses Riverpod for application state, GoRouter for navigation, Dio for API calls, and a charting component for analytics. The Python backend exposes authentication, upload/status, cabinet review, search, and export routes. Table I summarizes the implemented integration. The tested academic setup used a locally running backend, Flutter Web in a browser, an Android emulator, and configured Supabase/Gemini services.

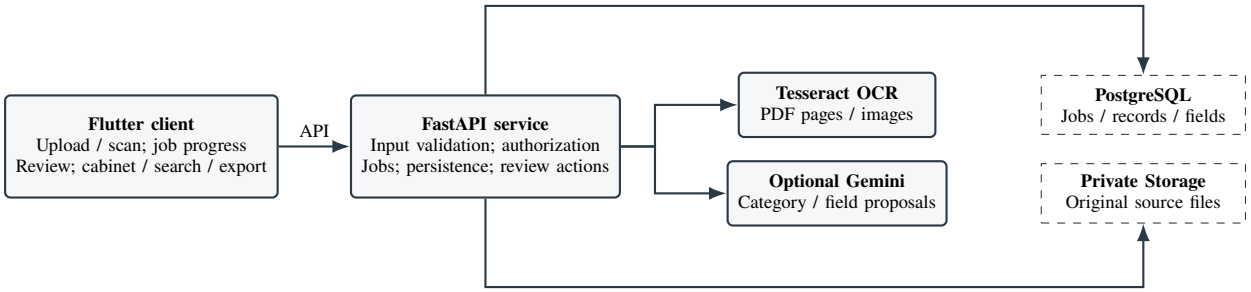


Fig. 1. DocSort logical architecture. Components and connections are redrawn from the project description and repository documentation; the diagram is not a measurement of deployment scale.

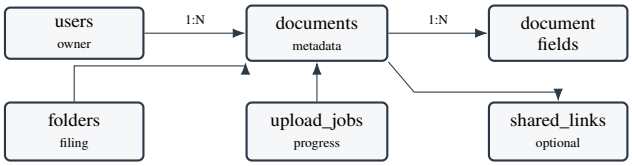


Fig. 2. Selected persistence entities. The schematic shows principal associations and omits secondary schema tables and column-level cardinality constraints.

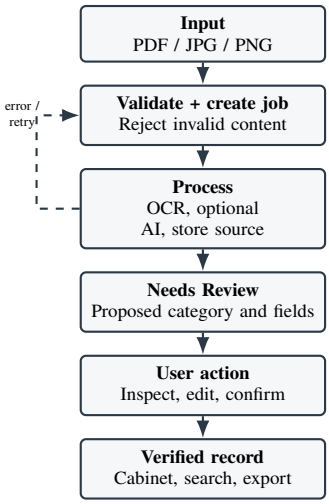


Fig. 3. Human-in-the-loop document lifecycle. "Processed" and "verified" are intentionally different outcomes.

B. Data Access and Integrity

The backend, rather than the client, applies access decisions to document and storage operations. Repository documentation describes hashed passwords, signed JWTs, live account-state checks, private storage, and integrity-sensitive export paths. It also describes safety controls for extensions and file signatures, bounded PDF processing, filename normalization, and CSV/ZIP exports [11]. These source-level measures are design safeguards, not proof of an external penetration test or security certification.

Because uploaded files can contain personal information, sensitive content should not be included in a public demonstration without consent. The optional AI step involves a network call to a third-party provider and therefore has a different privacy boundary from local Tesseract OCR. The prototype's backend-centric design centralizes this choice and provides an OCR/manual-review fallback; it does not, by itself, establish regulatory compliance or a comprehensive data-retention policy. The system should be deployed with explicitly configured credentials and a private storage bucket rather than embedding provider secrets in client code [9], [10].

The client retains a reviewable record, while the original file remains unchanged in private storage. This permits correcting metadata without modifying evidence of the uploaded source. Document analytics count saved application records and categories; such counts should not be read as independent estimates of AI accuracy.

C. Operational Boundaries

The upload handler delegates work to FastAPI background tasks. This provides asynchronous user feedback, but tasks are tied to the local server process and cannot be assumed to survive crashes. In production, a durable worker queue, recovery, and idempotent processing would be needed. Local concurrency controls and the current search approach also require reassessment at deployment scale. The project is therefore presented as an academic prototype, not an audited public service.

D. Design Tradeoffs and Failure Handling

Modular OCR and text interpretation offer a useful debugging boundary: if a category is wrong, the reviewer can distinguish an unreadable source, misrecognized text, and a mistaken downstream proposal. In exchange, stagewise processing can propagate OCR errors and may lose information about the original page layout. A layout-aware or OCR-free model could address different failure modes, but comparing those alternatives would require the same labeled inputs, hardware conditions, and evaluation criteria; the present results do not support such a ranking.

The upload job is also deliberately separate from the persistent document. A parser failure should produce a visible failed job, not a partially usable record. A temporarily unavailable Gemini service need not prevent local OCR and manual correction. Conversely, failure to retain the original source in private storage is not equivalent to failure of an optional AI enrichment step: a metadata record without retrievable source bytes would violate the file-to-record consistency requirement.

TABLE II
DOCUMENT PROCESSING RISKS AND DESIGNED RESPONSES

Condition	Expected system response
Invalid PDF/image	Reject before processing; show admission error.
Weak OCR / layout	Offer text/field proposal for correction; do not assert accuracy.
Wrong category	Keep record in *Needs Review* until confirmed.
AI service outage	Preserve OCR/manual review path.
Server crash	Jobs may not resume; durable queue is future work.

TABLE III
RECORD-STATE PROPERTIES AND THEIR EVIDENCE BOUNDARY

Property	Implementation or unresolved question
Review separation	A completed job is not automatically a verified document.
Source link	Private original and metadata are separate; hashes support integrity checks.
Authorization	Backend enforces document access; no external security audit.
Correction history	Current values can be edited; full immutable history is future work.
Durable processing	Background tasks work locally; crash-safe recovery is absent.

The release documentation describes backend cleanup and error paths for these situations, but their reliability has not been quantified under injected crashes and network faults [8], [11].

E. Retrieval as Part of Correctness

A document-understanding pipeline is useful only when its result can be found and safely delivered later. The Cabinet groups saved records, while Search uses document metadata and extracted text under authorized access. Export can deliver a structured CSV listing or a ZIP of permitted originals. In this context, end-to-end correctness includes more than label accuracy: the stored file must remain retrievable, a correction must persist, and inaccessible records must not leak through search or export. These are testable properties of the complete system and justify treating the cabinet as part of the research prototype rather than a cosmetic interface.

VI. Implementation Walkthrough and Reproducibility

A. A Document's Path Through the System

The following walkthrough describes the implemented sequence, not an additional experiment. A user selects a scan or PDF in Flutter, which sends the file to the upload endpoint. FastAPI checks the extension, actual file signature, parser readability, and configured admission limits. An accepted upload receives a job identifier, and the client polls for its status rather than waiting for the OCR operation inside one long HTTP response. For a multi-page PDF, the processing service renders the admitted pages and uses Tesseract to collect recognized text. The optional AI stage proposes a category and structured values; without a configured provider, the user can still rely on OCR and manual review.

The original file and its associated metadata are stored separately. The document then appears in a review interface where a person can accept or edit individual fields and the category. After explicit confirmation, the cabinet displays a verified record; the original source remains available through an authorized retrieval or export path. In the team's examples, an invoice or receipt can be organized under Finance, while a resume can be mapped to Human Resources. Those mappings are deterministic fallbacks described in the project materials, not evidence that the model predicts those types with any particular accuracy.

B. What a Successful Run Does and Does Not Show

A successful run establishes that the individual components can exchange data and that a reviewer can save a correction. It does not establish that every upload is interpreted correctly, that the file can always be recovered after a server crash, or that manual review is faster than entering fields from scratch. Even a displayed OCR confidence value is an internal score from the recognition stage rather than a measured percentage of correct names or dates. This distinction matters when a system combines machine suggestions with human-facing record states.

C. Repeatable Checks for a Future Replication

A repeatable implementation check should begin from a fresh checkout at a fixed commit, initialize the PostgreSQL schema and private storage bucket, configure Tesseract and optional provider credentials, and run the supplied backend and client test commands. The next stage should exercise an agreed set of PDFs and images under recorded conditions, save the returned job outcomes, and check both the reviewed record and its original source. Access-control scenarios should attempt unauthorized search, read, and export operations, while failure cases should deliberately include invalid files and unavailable dependencies. The existing repository contains tests and setup notes for these steps, but the paper does not claim that a fresh independent replication was performed.

VII. Validation and Observed Results

A. Evidence Sources

Two distinct evidence categories are available: repository-level regression reports and team-observed end-to-end demonstrations. The project's release documentation records 432 passing backend tests and 33 passing Flutter tests; these numbers are *reported by the team*, not newly executed or independently audited for this paper [11]. The project report describes manual browser and Android-emulator exercises with common document categories. The repository's later release notes also describe a Flutter Web release build and a browser smoke-test sequence; these are likewise author-reported checks.

TABLE IV
AVAILABLE VALIDATION EVIDENCE AND ITS LIMITATIONS

Evidence	Recorded outcome	Status
Backend test suite	432 passed in release notes	Reported
Flutter test suite	33 passed in release notes	Reported
Web/emulator demo	Upload, review, cabinet, search, export exercised	Manual
Sample documents	Most processed; some field and type errors	Qualitative
Labeled benchmark	No ground-truth set or measured scores	Absent

TABLE V
RECOMMENDED FUTURE EVALUATION OUTPUTS (NOT MEASURED)

Question	Evidence required
Classification	Ground-truth types; accuracy, macro-F1, confusion matrix.
Field extraction	Labeled key–value pairs; precision, recall, F1, exact match.
OCR text	Reference transcripts; CER and WER.
Review effort	Edits, review time, and field correction rate.
Response time	Page-aware processing and search latency; run environment.

B. Functional Observations

The key observed result is the completion of an end-to-end path: a document can be uploaded, processed, corrected, saved, searched, and exported. Team members reported that many ordinary samples yielded useful categories or fields, but some classifications were unsuitable and some fields missing or inaccurate. The review interface allowed these cases to be corrected, showing that the workflow remains usable when automated interpretation is imperfect.

This supports *functional feasibility*, not a population-level reliability or accuracy claim. A screen showing an OCR confidence score cannot establish that a date or name was extracted correctly. Similarly, an automated test using fixture data is evidence for the tested code contract, not evidence for document-understanding accuracy on unseen real-world material.

C. Reproducible Evaluation Protocol

To quantify future performance, a consented or synthetic set of documents should be labeled with ground-truth categories, transcripts, and target fields. A held-out test split should be fixed before measuring document-category accuracy, macro-F1, and confusion matrices. Field extraction should report entity-level precision, recall, F1, and exact match. OCR transcription should be compared against references using character error rate (CER) and word error rate (WER):

$$\mathrm{ER} = \frac{S + D + I}{N}, \tag{2}$$

where S, D, and I denote substitutions, deletions, and insertions, and N is the number of reference characters (CER) or words (WER). These are *proposed metrics*; no value is implied by Eq. (2).

The comparison should include three conditions using the same held-out document set: manual filename-and-folder organization; OCR and automatic metadata proposals without a correction stage; and the full DocSort workflow with review. The first condition measures the effort of ordinary filing, the second separates extraction errors from review behavior, and the third records whether corrections improve the final stored record. A fair comparison requires predeclared target fields, the same hardware and file quality, and consistent instructions to participants. Reviewers should not see ground-truth labels during their tasks. Results should be reported by document category and by scan quality, not solely as a pooled average.

For each document, retain its page count, source type, verified ground-truth category, target fields, raw OCR transcript, machine proposals, final accepted values, and whether the source can be retrieved. Distinguish wrong proposals corrected by a reviewer from wrong proposals accepted without change. That distinction allows the human-review step to be evaluated for its residual errors instead of being credited with accuracy simply because a confirmation button was pressed. *This paragraph describes a planned protocol; no participant study or comparative benchmark is reported here.*

A usability-oriented study should record the fraction of proposed fields edited, reviewer time per document, and successful retrieval after review. Processing latency should distinguish first-pass OCR from duplicate reuse, and report hardware, page counts, resolutions, network conditions, and representative percentiles. Access-control tests should include cross-user reads and failed export attempts. The repository provides evaluation utilities that could support such a study, but a labeled evaluation dataset and controlled trial remain outstanding [11].

VIII. Design Implications for Document Systems

The prototype illustrates why document processing has at least two forms of uncertainty. The first concerns recognition and interpretation: a word may be missed, a table may be flattened, or an amount may be associated with the wrong label. The second concerns record management: a prediction may be stored without anyone checking it, or a file may be available in a folder but inaccessible through authorized search. A better OCR model may improve the first issue without addressing the second. Keeping job state, confidence, and human verification separate makes their respective failure paths visible.

The modular architecture also makes the deployment trade-off explicit. Local OCR helps the system continue when the optional remote model is not configured; remote interpretation may yield useful structured suggestions but introduces availability and data-handling questions. Processing inside FastAPI background tasks simplifies the prototype, while a production system would need persisted job claims and retries. The same applies to search: a small cabinet can use the present bounded approach, but a large deployment would require query-plan

measurements and indexing choices supported by representative data.

Finally, human review should be tested rather than celebrated by default. A reviewer may catch a wrong category, miss an incorrect date, or spend longer checking the proposal than entering it manually. A useful evaluation would therefore report both the errors remaining after confirmation and the work required to reach that state. The relevant comparison is not only between OCR engines, but between complete record-creation workflows.

IX. Discussion and Limitations

The prototype's main practical strength is a reversible interpretation path: automation suggests metadata, but a person decides whether the result may become a verified record. Private original-file retention further separates source evidence from mutable interpretation. The component boundaries allow local OCR to continue when optional cloud AI is unavailable and keep privileged credentials on the server.

There are corresponding limitations. Poor image quality, multilingual material, tables, and unusual layouts can degrade OCR and downstream field extraction. The system has no independently measured recognition or classification accuracy. Manual review can correct individual records but creates human effort, and no automatic retraining effect has been demonstrated. The system has also not completed a production-scale load test, a formal security audit, or a crash-recovery study. Some optional administration, sharing, and People Map paths are implemented in the repository but are not the principal evaluated claims here.

The findings are therefore best interpreted as a carefully scoped *systems integration case study*. They do not establish superiority over research models or managed document-processing platforms, which operate under different data, accuracy, cost, and deployment assumptions.

A. Threats to Validity

The manual examples are a convenience sample rather than a random draw from a documented target population. Neither per-category denominators nor ground-truth transcription and field labels were preserved for a reproducible accuracy calculation. The reported regression counts come from repository release notes; their passing status depends on the tested revision and environment, and unit tests using fixtures do not exercise the same external services as a live upload. Since the primary backend ran on a development computer, performance cannot be extrapolated to mobile devices, independent servers, or multiple concurrent users. The absence of a blinded human-revision study also prevents claims that the interface improves review speed compared with manual filing. These limitations define the current inference boundary and the experiments needed to strengthen it.

X. Conclusion and Future Work

This work presents DocSort, a document-management prototype integrating PDF/image intake, Tesseract OCR, optional Gemini suggestions, private storage, PostgreSQL metadata, explicit user verification, and retrieval. The supplied project report and public implementation record a functioning workflow with correctable prediction errors. The absence of a labeled benchmark limits claims to functional demonstration and source-described validation. Priority next steps are a ground-truth evaluation set, repeatable OCR/field metrics, user correction-time studies, more robust small-screen layout, and a crash-durable processing queue. These would turn the present feasibility evidence into measurable reliability and performance conclusions.

Acknowledgment

The authors thank Dr. Jaykumar Shreeram Dhage, Project Mentor at the Institute of Information and Communication Technology, MGM University, for his guidance throughout the project.

OpenAI ChatGPT (GPT-6, accessed October 30, 2026) was used for manuscript drafting, language editing, LaTeX typesetting, and the conceptual diagrams in Figs. 1–3. The authors checked the technical statements against the project report and repository. No experimental data or performance plots were generated by the tool [12].

References

[1] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

[2] G. Jaume, H. K. Ekenel, and J.-P. Thiran, "FUNSD: A dataset for form understanding in noisy scanned documents," in *Proc. ICDAR Workshops*, 2019, doi: 10.1109/ICDARW.2019.10029.

[3] Y. Xu *et al.*, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. ACM SIGKDD*, 2020, pp. 1192–1200, doi: 10.1145/3394486.3403172.

[4] M. Li *et al.*, "TrOCR: Transformer-based optical character recognition with pre-trained models," *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 11, pp. 13094–13102, 2023, doi: 10.1609/aaai.v37i11.26538.

[5] G. Kim *et al.*, "OCR-free document understanding transformer," in *Computer Vision–ECCV 2022*, 2022, doi: 10.1007/978-3-031-19815-1_29.

[6] J. Fung *et al.*, "Human-in-the-loop technical document annotation: Developing and validating a system to provide machine-assistance for domain-specific text analysis," NIST Tech. Note 2287, 2024, doi: 10.6028/NIST.TN.2287.

[7] Flutter Team, "Build for and integrate with multiple platforms," Flutter Documentation. [Online]. Available: https://docs.flutter.dev/platform-integration. Accessed: Oct. 2026.

[8] FastAPI, "Background tasks," FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/tutorial/background-tasks/. Accessed: Oct. 2026.

[9] Supabase, "Storage buckets: Public and private buckets," Documentation. [Online]. Available: https://supabase.com/docs/guides/storage/buckets/fundamentals. Accessed: Oct. 2026.

[10] Google AI for Developers, "Structured outputs," Gemini API Documentation. [Online]. Available: https://ai.google.dev/gemini-api/docs/structured-output. Accessed: Oct. 2026.

[11] DocSort Project Team, "DocSort," version 1.0.0, public GitHub repository and project release documentation, 2026. [Online]. Available: https://github.com/N0tDeb/docsort. Accessed: Oct. 30, 2026.

[12] OpenAI, "ChatGPT (GPT-6)," software assistant, accessed Oct. 30, 2026. [Online]. Available: https://chatgpt.com/.